

Analysis of Autonomous Vehicle Navigation: Path Planning, Control, and Safety Integration

Prajjwal Dutta

School of Manufacturing System and Networks

Arizona State University

Tempe, Az, USA

pdutta9@asu.edu

Abstract—This report presents a systematic comparative study of autonomous ground vehicle navigation architectures that combine path planning, feedback control, safety filtering, and state estimation. Using a kinematic bicycle model in a $20\text{ m} \times 20\text{ m}$ static environment with obstacles, we explore the design space spanned by three planners (grid-based A*, sampling-based RRT*, and kinematic Hybrid A*), three controllers (PID, LQR, MPC), a control barrier function (CBF) safety layer, and five estimation modes (perfect state feedback, noisy odometry, EKF, UKF, and particle filter). The full combination yields $3 \times 3 \times 2 \times 5 = 90$ possible configurations in MATLAB that capture the main planner–controller–safety–estimation interactions. Performance is evaluated in terms of path length, tracking error, computation time, and safety violations. Results indicate that LQR achieves the lowest average tracking error, Hybrid A* produces kinematically feasible but more expensive paths, and the CBF layer eliminates collisions with modest impact on tracking and computation. The study provides quantitative guidelines for selecting planning, control, and safety modules for real-time autonomous navigation in cluttered environments.

Index Terms—autonomous vehicles, path planning, model predictive control, PID control, LQR, control barrier functions, Kalman filtering, particle filtering

I. INTRODUCTION

Autonomous ground and marine vehicles increasingly rely on tightly integrated navigation stacks that combine global path planning, feedback control, safety filtering, and state estimation. In practical deployments, these modules are rarely used in isolation: a planner must generate dynamically feasible paths around obstacles, a controller must track these paths under modelling error and disturbances, a safety layer must override unsafe control inputs, and the overall system must operate on imperfect state estimates produced by external sensors and filters [1]–[3]. Understanding how these elements interact—and how design choices in one layer affect the others—is critical for building reliable autonomous systems.

This report documents a simulation assignment carried out as part of a graduate-level course on autonomous vehicle navigation. The goal of the assignment is not to propose new algorithms, but to implement a representative set of classical methods and to compare their behaviour in a controlled environment. Specifically, we construct a modular MATLAB and ROS 2 simulation framework for a planar autonomous

vehicle model, and we use it to study the interaction between three classes of algorithms:

- Path planners: grid-based A*, sampling-based RRT*, and a kinematically-feasible Hybrid A* planner.
- Feedback controllers: PID, Linear Quadratic Regulator (LQR), and Model Predictive Control (MPC).
- Safety and estimation layers: a Control Barrier Function (CBF) based safety filter and multiple state-estimation modes (perfect state knowledge, noisy ground truth, Extended Kalman Filter (EKF), Unscented Kalman Filter (UKF), and Particle Filter (PF)).

The vehicle is modelled using a bicycle/unicycle kinematics with state

$$x = [x, y, \theta, v]^T,$$

where $(x, y) \in \mathbb{R}^2$ is the position in meters, $\theta \in \mathbb{R}$ is the heading in radians, and $v \in \mathbb{R}$ is the linear speed. The control input is

$$u = [v_c, \omega]^T,$$

where v_c is the commanded linear velocity and ω is the commanded angular velocity. This model is standard in mobile robotics and captures the nonholonomic constraint while remaining simple enough for efficient simulation and controller design [4]. All experiments are conducted in a $20\text{ m} \times 20\text{ m}$ environment with static circular obstacles, a fixed start–goal pair, and a uniform sampling time of $\Delta t = 0.1\text{ s}$.

From a pedagogical standpoint, the assignment emphasises three aspects:

- 1) **Modular software architecture.** The planning, control, safety, and estimation modules are implemented as interchangeable components in MATLAB and as separate ROS 2 nodes. This mirrors typical autonomy stacks where modules are developed and tested independently, then integrated through well-defined interfaces.
- 2) **Algorithmic trade-offs.** By using canonical algorithms for each layer (A*, RRT*, Hybrid A* [1], PID/LQR/MPC [5], [6], and CBF-based safety filters [7]), the focus shifts from novelty to understanding trade-offs in path quality, tracking accuracy, computational cost, and safety.
- 3) **Impact of state estimation.** The main autonomy loop accepts state estimates as an external input. Within this

This work was carried out as part of a graduate-level robotics and controls project.

framework we explore how different estimation qualities (perfect, noisy, EKF, UKF, PF) affect closed-loop behaviour without modifying the downstream planning and control logic.

In principle, the full design space includes 90 possible configurations. Running and documenting all 90 combinations would add little insight relative to the effort required. Instead, this report focuses on a curated subset of eighteen configurations that are representative of the main qualitative behaviours. These configurations are chosen to highlight: (i) differences between grid-based and sampling-based planning, (ii) the effect of controller sophistication on tracking performance, (iii) the influence of the CBF layer on safety and control effort, and (iv) the sensitivity of the overall system to estimation quality.

The work described here is closely aligned with current research trends that combine optimization-based control and safety certificates for autonomous systems. Control Barrier Functions provide a convenient way to encode collision-avoidance and connectivity constraints as forward-invariant safe sets [7], [8], and to enforce them via real-time quadratic programs with minimal deviation from a nominal controller. Similarly, MPC has become a standard tool for constrained trajectory tracking in both ground and marine vehicles [6], [9], while A* and RRT*-style planners remain the workhorses of motion planning libraries in practice [1]. By integrating these methods in a common test bench, the assignment offers a small-scale but realistic case study of how such techniques behave when combined.

The remainder of this report is organised as follows. Section II reviews relevant literature on path planning, feedback control, safety frameworks based on control barrier functions, and state estimation methods for mobile robots. Section III describes the overall methodology, including the system architecture, mathematical formulation of the planners, controllers, CBF safety filter, and estimator models, as well as the experimental setup. Section IV summarises key implementation details in MATLAB and ROS 2. Section V presents the simulation results for the selected eighteen configurations, including quantitative metrics and qualitative trajectory plots. Section VI discusses the main observations, trade-offs, and limitations of the current framework, and Section VII concludes with recommendations and directions for future work.

II. SIMULATION ENVIRONMENT AND SOFTWARE ARCHITECTURE

The assignment requires a MATLAB-based simulation and a minimal but functional ROS 2 package that implement the same navigation stack (planning, control, CBF safety layer) and accept external state estimates rather than computing them internally. This section summarizes the architecture of both environments and how they interact with the common vehicle and environment models.

MATLAB Navigation Architecture (Planning–Control–CBF–Estimation)

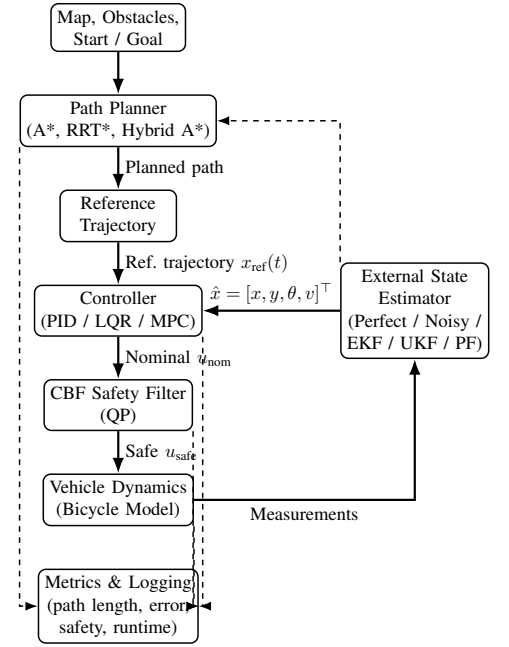


Fig. 1. Overall navigation architecture. The MATLAB simulation accepts external state estimates (perfect, noisy, EKF, UKF, PF), performs path planning, closed-loop control (PID/LQR/MPC), and safety filtering via a Control Barrier Function (CBF) QP before applying inputs to the bicycle model. Metrics are logged for comparative analysis.

A. MATLAB Environment

1) *Overall simulation architecture:* The MATLAB implementation is organized as a modular navigation stack (Fig. 1). A top-level script configures the experiment and then executes a main loop with the following steps:

- 1) **Environment setup:** construct an `Environment` instance, set the map bounds and resolution, and call a scenario generator (e.g., `create_simple_scenario`) to populate static obstacles and define start/goal poses.
- 2) **Module initialization:** instantiate `VehicleDynamics` (bicycle model), a planner (`AStarPlanner`, `RRTStarPlanner`, `HybridAStarPlanner`), a controller (`PIDController`, `LQRController`, `MPCController`), and optionally a `CBFSafetyFilter`.
- 3) **State-estimation mode:** select an estimator mode (perfect, noisy, EKF, UKF, or PF) and associated noise parameters; the main loop only consumes state estimates $\hat{x}$ from this module and does not implement estimation inside the stack.
- 4) **Global planning:** given the current state and goal, call the chosen planner once to generate a collision-free path in the `Environment` (Section II-C). The path is then time-parameterized into a reference trajectory $x_{\text{ref}}(t)$.
- 5) **Closed-loop execution:** at each time step k :
 - a) receive $\hat{x}_k$ from the estimator mode;
 - b) query the controller with $(\hat{x}_k, x_{\text{ref}})$ to obtain a

- nominal velocity command $(v_{\text{nom},k}, w_{\text{nom},k})$;
- c) pass the nominal command through the CBF safety filter (if enabled) to obtain $(v_{\text{safe},k}, w_{\text{safe},k})$;
- d) apply $(v_{\text{safe},k}, w_{\text{safe},k})$ to `VehicleDynamics` and update the true state x_{k+1} ;
- e) log states, controls, distances to obstacles, and timing information for later analysis.

This loop realizes the required pipeline state input $\rightarrow$ planning $\rightarrow$ CBF safety refinement $\rightarrow$ control $\rightarrow$ vehicle dynamics, while keeping state estimation external as specified in the assignment.

2) *Vehicle model*: The `VehicleDynamics` class supports two planar models: (i) a kinematic bicycle model with state

$$x = [x, y, \theta, v]^\top, \quad (1)$$

and (ii) a differential-drive model with state

$$x = [x, y, \theta]^\top. \quad (2)$$

All simulations in this report use the bicycle model; the differential-drive mode is included for potential extensions.

For the bicycle model, the control input is

$$u = [a, \delta]^\top, \quad (3)$$

where a is longitudinal acceleration and δ is the steering angle. The continuous-time kinematics are

$$\dot{x} = v \cos \theta, \quad (4)$$

$$\dot{y} = v \sin \theta, \quad (5)$$

$$\dot{\theta} = \frac{v}{L} \tan \delta, \quad (6)$$

$$\dot{v} = a, \quad (7)$$

with wheelbase L . The implementation uses an explicit Euler discretization with fixed time step

$$\Delta t = 0.1 \text{ s}. \quad (8)$$

Given $x_k = [x_k, y_k, \theta_k, v_k]^\top$ and $u_k = [a_k, \delta_k]^\top$, inputs are first saturated,

$$a_k^{\text{sat}} = \text{sat}_{[-a_{\text{max}}, a_{\text{max}}]}(a_k), \quad (9)$$

$$\delta_k^{\text{sat}} = \text{sat}_{[-\delta_{\text{max}}, \delta_{\text{max}}]}(\delta_k), \quad (10)$$

then the speed and pose are updated as

$$v_{k+1} = \text{sat}_{[0, v_{\text{max}}]}(v_k + a_k^{\text{sat}} \Delta t), \quad (11)$$

$$x_{k+1} = x_k + v_k \cos \theta_k \Delta t, \quad (12)$$

$$y_{k+1} = y_k + v_k \sin \theta_k \Delta t, \quad (13)$$

$$\theta_{k+1} = \text{wrapToPi}\left(\theta_k + \frac{v_k}{L} \tan(\delta_k^{\text{sat}}) \Delta t\right). \quad (14)$$

Default parameters in the MATLAB simulator are $L = 2.5 \text{ m}$, $v_{\text{max}} = 5.0 \text{ m/s}$, $a_{\text{max}} = 2.0 \text{ m/s}^2$, $\delta_{\text{max}} = \frac{\pi}{4} \text{ rad}$. The same kinematic structure is used in the ROS 2 simulator node, with numerical values loaded from `params.yaml`.

3) *Environment, scenarios, and noise*: The `Environment` class encapsulates the 2D map, start/goal positions, and obstacle geometry. The map is parameterized by

$$\text{map_size} = [W, H], \quad (15)$$

representing width and height in meters, and by a scalar grid resolution `resolution` used by grid-based planners. For the experiments in this report, we use

$$W = H = 20 \text{ m}, \quad \text{resolution} = 0.5 \text{ m/cell}, \quad (16)$$

so that the workspace matches the $20 \times 20 \text{ m}$ requirement in the assignment. The start and goal are stored as

$$\text{start_pos} = [x_{\text{start}}, y_{\text{start}}], \quad \text{goal_pos} = [x_{\text{goal}}, y_{\text{goal}}], \quad (17)$$

and are held fixed across all configurations to enable fair comparison.

Obstacles are stored as a cell array of structs with a `type` field and geometric parameters:

- circles: `type = 'circle'`, center (x_j, y_j) , radius R_j ;
- rectangles: `type = 'rectangle'`, bottom-left (x_j, y_j) , width w_j , height h_j .

Methods such as `add_circular_obstacle` and `add_rectangular_obstacle` build scenarios programmatically; a helper `create_simple_scenario` sets a canonical test setup with a few obstacles positioned to create narrow passages between start and goal.

Collision checks are performed by `check_collision(x, y, safety_margin)`, which returns `true` if (x, y) is outside the map or lies inside any obstacle inflated by a safety margin d_{safe} :

- circular obstacles: $\|[x, y]^\top - p_j\|_2 < R_j + d_{\text{safe}}$;
- rectangular obstacles: inside a rectangle expanded by d_{safe} in all directions.

A path-level function `check_path_collision` applies this test to all samples along a candidate path and is used both by the planners and by the CBF layer.

State-estimation noise is injected by adding Gaussian perturbations to position and heading (noisy mode) or by using precomputed $\{\hat{x}_k\}$ from EKF/UKF/PF logs. In all cases, the navigation modules receive only $\hat{x}_k$.

4) *Logging and metrics*: During each MATLAB simulation, the framework logs:

- the planned path and the executed trajectory;
- the reference trajectory $x_{\text{ref},k}$ used by the controller;
- nominal and safe control commands $(v_{\text{nom},k}, w_{\text{nom},k})$ and $(v_{\text{safe},k}, w_{\text{safe},k})$;
- the minimum distance to the closest obstacle at each time step;
- planner statistics (path length, planning time, number of expanded nodes, success flag);
- tracking metrics (RMS error, maximum error);
- timing (average control computation time, CBF QP time).

These are exported as `.csv` and `.mat` files and used to construct the comparison tables and plots in the Results section.

B. ROS2 Environment

The ROS 2 environment mirrors the same architecture in a node- and topic-based framework. All navigation functionality is contained in a single Python package `vehicle_nav` inside a ROS 2 workspace, with structure summarized in `FILE_MANIFEST.md`.

1) *Package structure and nodes*: The package layout is:

```
ros2_ws/src/vehicle_nav/  
  vehicle_nav/  
    simulator_node.py  
    astar_node.py  
    rrtstar_node.py  
    pid_node.py  
    lqr_node.py  
    mpc_node.py  
    cbf_node.py  
    goal_publisher.py  
    metrics_node.py  
  launch/  
    sim_launch.py  
  config/  
    params.yaml  
    vehicle_nav.rviz  
package.xml  
setup.py
```

The main nodes used in this report are:

- **simulator_node.py**: implements the bicycle kinematics, publishes the vehicle state and obstacles, and accepts `/cmd_vel_safe` (or `/cmd_vel_nom` if CBF is disabled) as control input.
- **astar_node.py** and **rrtstar_node.py**: A* and RRT* global planners subscribing to `/state` and `/goal` and publishing `/planned_path`.
- **pid_node.py**, **lqr_node.py**, **mpc_node.py**: controller nodes that subscribe to `/state` and `/planned_path` and publish nominal commands on `/cmd_vel_nom`.
- **cbf_node.py**: CBF safety filter that subscribes to `/state`, `/cmd_vel_nom`, and `/obstacles` and publishes `/cmd_vel_safe`.
- **goal_publisher.py**: convenience node to publish `/goal` from the command line.
- **metrics_node.py**: collects performance metrics and writes JSON summaries (path length, tracking error, safety, runtime).

2) *Topic architecture and messages*: The topic graph is summarized in `TOPIC_ARCHITECTURE.md` and replicated in Fig. 2. The key topics are:

- `/state` (PoseStamped): current vehicle pose in the map frame, published by the simulator at 50Hz and consumed by planners, controllers, and the CBF.
- `/goal` (PoseStamped): goal pose, published by the user or `goal_publisher`.
- `/planned_path` (Path): global path published by the active planner and consumed by all controllers.

- `/cmd_vel_nom` (Twist): nominal linear and angular velocity commands from the active controller to the CBF node.
- `/cmd_vel_safe` (Twist): safety-filtered commands from the CBF node to the simulator; equals `/cmd_vel_nom` when CBF is disabled.
- `/obstacles` (MarkerArray): static obstacle markers published by the simulator and used by the CBF constraints; also visualized in RViz.

Quality-of-service (QoS) policies follow standard conventions: state and control topics are RELIABLE/KEEP_LAST(1), while path and obstacle topics are RELIABLE/TRANSIENT_LOCAL to allow late subscribers to see the latest path or map.

3) *Launch setup and parameters*: The main launch file `launch/sim_launch.py` starts the full navigation stack with configurable planner, controller, and CBF options. Key launch arguments are:

- `planner`: `astar` or `rrtstar`;
- `controller`: `pid`, `lqr`, or `mpc`;
- `use_cbf`: `true` or `false`;
- `use_rviz`: `true` or `false`.

Depending on these arguments, the launch file starts the simulator node, the selected planner and controller node, the CBF node (if enabled), and RViz 2 (if requested).

All algorithm and model parameters are stored in `config/params.yaml`, including:

- vehicle parameters (wheelbase, maximum speed, maximum angular velocity);
- environment bounds and obstacle layout;
- planner parameters (A* grid resolution, RRT* step size, iterations, search radius, safety margin);
- controller gains (PID, LQR Q/R , MPC horizon and costs);
- CBF parameters (safety margin, α , repulsive gains).

4) *Assumptions and simplifications*: The ROS 2 environment makes the same modeling assumptions as the MATLAB simulator:

- 2D planar motion with a kinematic bicycle model;
- static obstacles represented as markers in the map frame;
- state estimates provided externally as `/state` (PoseStamped) rather than derived from raw sensors;
- a single global path at a time, with controllers performing local tracking based on lookahead distance.

RViz visualization is configured via `vehicle_nav.rviz`, but in practice some displays (paths, markers) must be added manually and the fixed frame must be set to `map` before saving the configuration.

C. Path Planning Algorithms

1) *Grid-Based A* Path Planning*: For global path planning, we use a standard grid-based A* algorithm implemented in the `AStarPlanner` class. The continuous 2D workspace defined by the `Environment` object is discretized into a uniform occupancy grid with user-defined resolution Δ_{grid} .

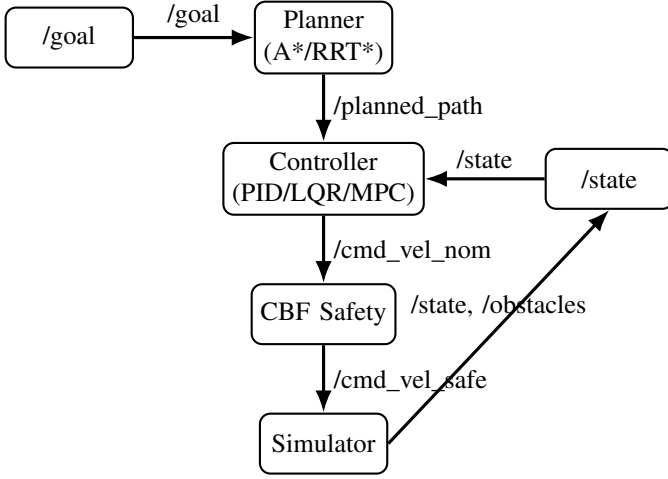


Fig. 2. ROS2 topic architecture for the navigation stack. The planner publishes a global path, the controller publishes nominal velocity commands, the CBF node filters them for safety, and the simulator node publishes the current state and obstacles.

(m/cell). The start and goal positions are given in world coordinates and mapped to grid indices by rounding; cells that lie inside any obstacle or outside the map bounds, as reported by `Environment.check_collision`, are treated as occupied and cannot be traversed.

The search is carried out on an 8-connected grid (horizontal, vertical, and diagonal moves). Each grid cell (i, j) corresponds to a node with cost-to-come $g(i, j)$ and heuristic $h(i, j)$ to the goal. The edge cost between neighboring cells is the Euclidean distance in grid space, scaled by the grid resolution, so axis-aligned moves cost Δ_{grid} and diagonal moves cost $\sqrt{2}\Delta_{\text{grid}}$. The heuristic h is chosen as the Euclidean distance to the goal in meters; in preliminary tests we also evaluated Manhattan and diagonal (Chebyshev) heuristics, but Euclidean provided the best trade-off between path quality and number of expansions and is used in all experiments. The total score is

$$f(i, j) = g(i, j) + h(i, j), \quad (18)$$

and at each iteration A* expands the open node with minimum f .

The algorithm terminates when the goal cell is popped from the open set or the open set becomes empty (no path found). If a path is found, the planner backtracks parent pointers from goal to start, then converts the resulting grid sequence back to world coordinates to obtain a polyline

$$\text{path} = \{[x_k, y_k]^T\}_{k=0}^N$$

expressed in meters. The geometric path length

$$L = \sum_{k=0}^{N-1} \|[x_{k+1}, y_{k+1}]^T - [x_k, y_k]^T\|_2 \quad (19)$$

is recorded as one of the performance metrics and later compared to the paths produced by RRT* and Hybrid A*.

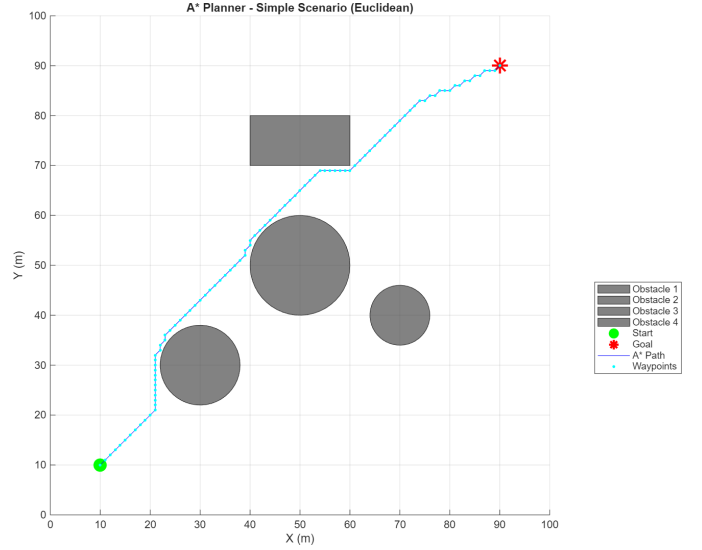


Fig. 3. Grid-based A* path in the simple scenario using the Euclidean heuristic. Obstacles are shown in gray, the start and goal positions are marked in green and red, respectively, and the planned path is plotted as a polyline connecting the grid waypoints.

Pseudocode (A*):

- 1) Initialize the open set with the start node ($g = 0$, h from start to goal).
- 2) While the open set is not empty:
 - a) Select the node with the smallest $f = g + h$.
 - b) If this node is the goal, reconstruct the path by following parent pointers and terminate.
 - c) Move the node to the closed set.
 - d) For each 8-connected neighbor:
 - Skip if out of bounds, in collision, or already in the closed set.
 - Compute tentative $g_{\text{new}} = g_{\text{current}} + \text{move_cost}$.
 - If the neighbor is not in the open set, or g_{new} is smaller than its stored g , update g , recompute h and f , set the parent, and insert into the open set.

Complexity: On a grid with N free cells, A* with a priority queue has time complexity $O(N \log N)$ and space complexity $O(N)$. Our MATLAB implementation uses a simple linear search over the open set instead of a heap, so the practical worst-case complexity grows closer to $O(N^2)$, but this remains tractable for the map sizes considered.

Figure 3 illustrates a typical A* path in the simple scenario, showing how the grid-based planner generates a collision-free polyline around circular and rectangular obstacles.

2) Sampling-Based RRT* Path Planning: The second planner is a sampling-based RRT* algorithm implemented in the `RRTStarPlanner` class. RRT* builds a tree of continuous states in the (x, y) workspace defined by the `Environment` object. Each node stores its position, parent index, and accumulated cost-to-come.

At each iteration k , the planner samples a random point $x_{\text{rand}} \in \mathbb{R}^2$ uniformly inside the map; with probability

`goal_sample_rate = 0.1` the goal position is sampled instead to bias the search. The nearest existing node x_{near} in the tree is found in Euclidean distance, and a new state x_{new} is generated by steering from x_{near} towards x_{rand} with a fixed step size `step_size` (default 2 m). The straight-line segment from x_{near} to x_{new} is collision-checked by sampling intermediate points at 0.5 m spacing and calling `Environment.check_collision`; only collision-free candidates are inserted.

RRT* optimality is implemented through a local rewiring step. All existing nodes within a radius `search_radius = 5 m` of x_{new} are collected. Among these “near” nodes, the algorithm selects as parent the one that yields the smallest cost-to-come while remaining collision-free. After insertion, neighboring nodes are potentially rewired through x_{new} if their path cost can be reduced without introducing collisions. When x_{new} lies within a goal threshold `goal_threshold = 1.5 m` of the goal, the corresponding node index and total cost are stored as the current best solution. The loop runs until either a good solution is found (with early termination after sufficient iterations) or a maximum number of iterations `max_iterations` is reached.

If a goal node has been found, the final path is obtained by backtracking parent pointers from the best goal node to the root and prepending the exact goal position, yielding a polyline

$$\text{path} = \{[x_k, y_k]^\top\}_{k=0}^N$$

in world coordinates. As with A*, the geometric path length

$$L = \sum_{k=0}^{N-1} \|[x_{k+1}, y_{k+1}]^\top - [x_k, y_k]^\top\|_2 \quad (20)$$

is recorded together with summary statistics (iterations, number of nodes, runtime, success flag) and later compared against the A* and Hybrid A* planners.

Pseudocode (RRT):*

- 1) Initialize the tree with the start node.
- 2) For $k = 1$ to `max_iterations`:
 - a) Sample x_{rand} uniformly (or the goal with probability `goal_sample_rate`).
 - b) Find the nearest node x_{near} in the tree.
 - c) Steer from x_{near} toward x_{rand} to get x_{new} ; discard if the segment is in collision.
 - d) Find all near nodes within `search_radius`.
 - e) Choose as parent the near node that yields the lowest collision-free cost-to-come, and add x_{new} .
 - f) Rewire near nodes through x_{new} if this reduces their cost without collisions.
 - g) If x_{new} is within `goal_threshold` of the goal, update the best goal node.

Complexity: For K iterations and a naive linear nearest-neighbor search, the time complexity of our implementation is $O(K^2)$ and the memory usage is $O(K)$ (one node per iteration). With more advanced nearest-neighbor data structures, RRT* can be reduced to approximately $O(K \log K)$, but this was unnecessary for the map sizes in this assignment.

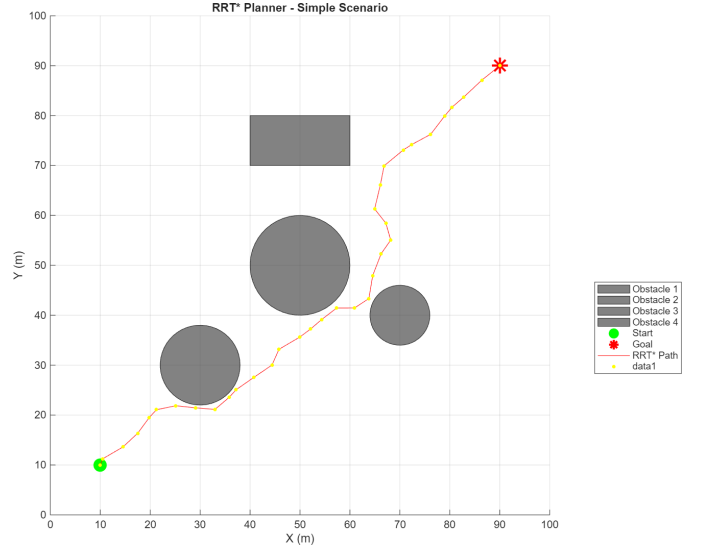


Fig. 4. RRT* path in the simple environment. Random samples grow a tree from the start toward the goal while avoiding circular and rectangular obstacles. The final path (highlighted polyline) is obtained by backtracking from the best goal node after the rewiring step.

Figure 4 illustrates a typical RRT* solution in the simple scenario, showing the continuous path obtained after rewiring and backtracking through the tree.

3) *Hybrid A* Path Planning:* The third planner is a Hybrid A* algorithm implemented in the `HybridAStarPlanner` class. In contrast to grid-based A*, Hybrid A* searches directly in the continuous state space

$$x = [x, y, \theta]^\top,$$

while still using a grid to discretize position and heading for the search. The map is discretized with the same spatial resolution Δ_{grid} as A*, and the heading is discretized into `num_headings = 72` bins (5° per bin). Each state is keyed as integer indices (i, j, k) obtained by rounding x/Δ_{grid} , y/Δ_{grid} and $\theta/\Delta\theta$, and stored in `containers.Map` open/closed sets.

From each expanded node, successor states are generated using simple vehicle motion primitives that approximate the kinematic bicycle model. The `get_successors` method applies three primitives: straight, left turn, and right turn, with steering angles $\{0, \pm 30^\circ\}$ and a fixed step length of $2 \Delta_{\text{grid}}$. Each primitive integrates a short arc of motion with a fixed turning radius `turning_radius = 5 m`, updating (x, y, θ) accordingly and wrapping θ to $(-\pi, \pi]$.

Each candidate state is rejected if it is in collision according to `check_state_collision`. This routine checks both the vehicle center (as a circle of radius $L/2$) and a rectangular footprint of length L and width $L/3$ rotated by the current heading, by calling `Environment.check_collision` on the center and the four corners. The goal is considered reached when the position error is less than $2 \Delta_{\text{grid}}$ and the heading error is below 30° .

Hybrid A* uses a weighted A* cost

$$f = g + \varepsilon h, \quad (21)$$

with $\varepsilon = \text{heuristic_weight} = 2.0$ to accelerate search at the cost of mild suboptimality. The cost-to-come g is the geometric path length accumulated along motion primitives, and the heuristic h is the Euclidean distance in (x, y) to the goal. The planner expands the node with minimum f until the goal condition is satisfied or a limit on the number of expanded nodes is reached. If successful, the final path is obtained by backtracking parent pointers, yielding a sequence

$$\text{path} = \{[x_k, y_k, \theta_k]^T\}_{k=0}^N,$$

and the executed path length $L = \sum_{k=0}^{N-1} \|[x_{k+1}, y_{k+1}]^T - [x_k, y_k]^T\|_2$ is stored in the statistics struct for later comparison.

Pseudocode (Hybrid A):*

- 1) Initialize the open set with the discretized start state.
- 2) While the open set is not empty:
 - a) Select the state with the smallest $f = g + \varepsilon h$.
 - b) If the position and heading are within the goal tolerances, reconstruct the path and terminate.
 - c) Move the state to the closed set.
 - d) Generate successor states using straight/left/right motion primitives; discard successors in collision or already closed.
 - e) For each valid successor, compute new g , h , and f , set the parent, and insert into the open set if it improves the existing entry.

Complexity: Hybrid A* performs A*-style search on a 3D grid (i, j, k) of position and heading. If N states are expanded, the ideal complexity with a priority queue is $O(N \log N)$ and memory usage is $O(N)$. As with the A* implementation, our MATLAB code uses a simple linear search over the open set, so the empirical complexity is closer to $O(N^2)$, but the search remains tractable for the map sizes and discretization used in this assignment.

D. Control Strategies

1) *PID Trajectory Tracking Controller:* The baseline feed-back controller is a PID-based trajectory tracker implemented in the `PIDController` class. The controller operates on the vehicle state

$$x = [x, y, \theta, v]^T$$

and a desired state

$$x_d = [x_d, y_d, \theta_d, v_d]^T$$

and outputs a commanded linear velocity v_{cmd} and angular velocity w_{cmd} .

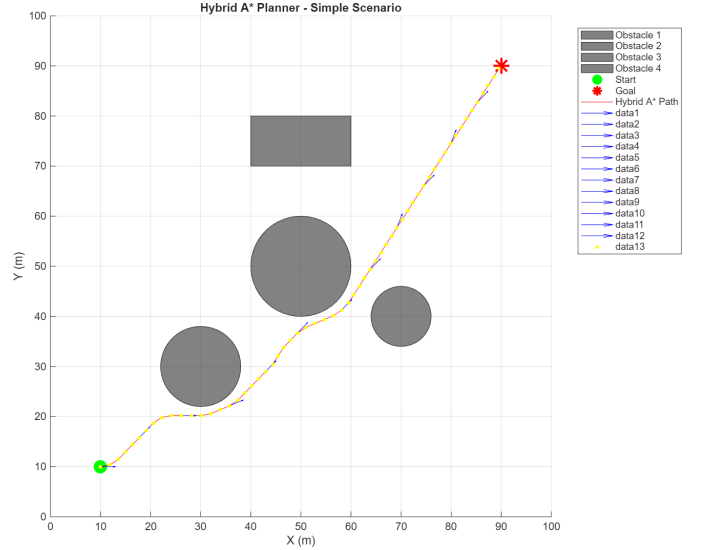


Fig. 5. Hybrid A* path in the simple environment. The planner searches in (x, y, θ) using motion primitives (straight and curved arcs) that respect the vehicle's minimum turning radius, and performs collision checking on a finite rectangular footprint.

Control law: Given the current state x and desired state x_d , the controller first computes the position error in the world frame

$$dx = x_d - x, \quad dy = y_d - y,$$

then transforms it to the vehicle frame:

$$e_x = \cos \theta dx + \sin \theta dy, \quad (22)$$

$$e_y = -\sin \theta dx + \cos \theta dy. \quad (23)$$

The distance to the target and the heading error are

$$\text{dist} = \sqrt{dx^2 + dy^2}, \quad (24)$$

$$e_\theta = \text{atan2}(\sin(\theta_d - \theta), \cos(\theta_d - \theta)), \quad (25)$$

with $e_\theta \in [-\pi, \pi]$. The desired speed is chosen as

$$v_{\text{des}} = \begin{cases} \min(v_d, \text{dist}), & \text{dist} > 0.1, \\ 0, & \text{otherwise,} \end{cases}$$

so the vehicle slows down as it approaches the target.

The velocity tracking error is

$$e_v = v_{\text{des}} - v,$$

and the controller maintains integral and derivative terms:

$$I_v \leftarrow \text{sat}_{[-1,1]}(I_v + e_v \Delta t), \quad (26)$$

$$I_w \leftarrow \text{sat}_{[-1,1]}(I_w + e_\theta \Delta t), \quad (27)$$

$$D_v = \frac{e_v - e_v^{\text{prev}}}{\Delta t}, \quad (28)$$

$$D_w = \frac{e_\theta - e_\theta^{\text{prev}}}{\Delta t}, \quad (29)$$

where Δt is the time step and $\text{sat}_{[-1,1]}(\cdot)$ denotes anti-windup saturation.

The PID control outputs are

$$v_{\text{PID}} = K_p^v e_v + K_i^v I_v + K_d^v D_v, \quad (30)$$

$$w_{\text{PID}} = K_p^w e_\theta + K_i^w I_w + K_d^w D_w, \quad (31)$$

with gains (K_p^v, K_i^v, K_d^v) for speed and (K_p^w, K_i^w, K_d^w) for heading. A simple feedforward term adds the desired speed:

$$v_{\text{cmd}} = v_{\text{PID}} + v_{\text{des}},$$

and the commands are finally saturated:

$$v_{\text{cmd}} \leftarrow \max(0, \min(v_{\text{cmd}}, v_{\text{max}})), \quad (32)$$

$$w_{\text{cmd}} \leftarrow \max(-w_{\text{max}}, \min(w_{\text{cmd}}, w_{\text{max}})), \quad (33)$$

with $v_{\text{max}} = 5.0$ m/s and $w_{\text{max}} = \pi/2$ rad/s in the implementation. The previous errors e_v^{prev} and e_θ^{prev} are stored for the next time step.

Tuning strategy.: The controller constructor accepts explicit gains, with default values

$$K_p^v = 1.0, K_i^v = 0.1, K_d^v = 0.5,$$

$$K_p^w = 2.0, K_i^w = 0.1, K_d^w = 0.3.$$

These were used as initial guesses and refined empirically by running closed-loop simulations on the simple A* path scenario. The tuning procedure was:

- Increase K_p^v until the heading converges quickly without excessive overshoot.
- Adjust K_d^v to damp oscillations in e_θ .
- Tune K_p^v and K_d^v so that the speed follows v_{des} without large overshoot or sluggish response.
- Introduce small integral gains K_i^v and K_i^w to remove steady-state errors due to modeling errors, while keeping the integrator saturation at ± 1 to avoid windup.

The final gains provide a compromise between tracking accuracy and control smoothness across all tested paths.

Stability considerations.: A formal Lyapunov stability proof is beyond the scope of this assignment. However, for small tracking errors and without saturation, the linearized error dynamics in (e_v, e_θ) with positive proportional gains are locally stabilizing: $K_p^v > 0$ drives the heading error toward zero, while $K_p^w > 0$ regulates the speed toward v_{des} . The derivative terms add damping, and the integral terms compensate steady-state bias. In practice, the combination of bounded integrators, rate-limited updates (via Δt), and input saturation prevents runaway behavior; all closed-loop simulations with the tuned gains remained stable and converged to the goal region.

Trajectory tracking mechanism.: To track a full path $\{[x_k, y_k, \theta_k]\}_{k=0}^N$ produced by the planner, the method `track_path` implements a pure-pursuit-style lookahead:

- For the current state $[x, y, \theta, v]$, find the closest point on the path in Euclidean distance.
- Starting from this index, move forward along the path until the distance to the vehicle is at least a specified lookahead distance ℓ_{LA} ; this point becomes the lookahead target.

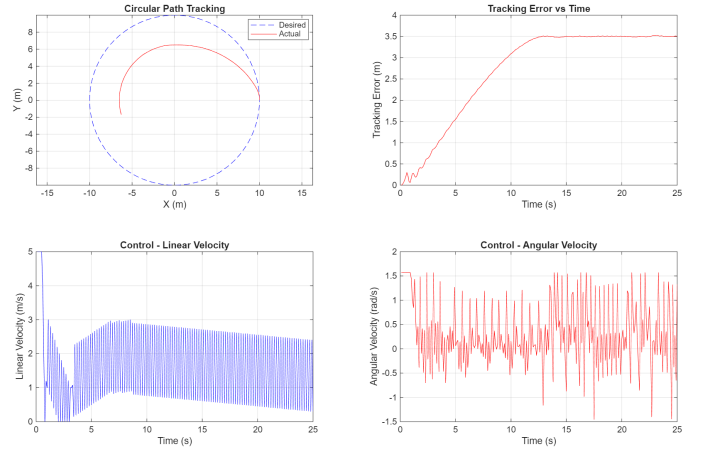


Fig. 6. PID controller performance on a circular path tracking experiment. Top left: desired (dashed) and actual (solid) trajectories. Top right: tracking error over time. Bottom: commanded linear and angular velocities. The plot illustrates the baseline tracking capability of the PID controller as well as its oscillatory behavior when following a curved path.

- Construct a desired state $x_d = [x_d, y_d, \theta_d, v_d]^\top$ from the lookahead point (with a nominal v_d), and pass (x, x_d) to `compute_control`.

In this way, the PID controller always tracks a point slightly ahead of the vehicle rather than the closest path point, which improves smoothness and robustness to discretization of the planned trajectory. The same PID implementation is used with A*, RRT*, and Hybrid A* paths; only the input path changes, not the control law.

Figure 6 shows a representative circular path tracking experiment, highlighting the steady-state tracking error and the relatively high-frequency oscillations in the commanded velocities that motivate the comparison with LQR and MPC in later sections.

2) *LQR Trajectory Tracking Controller*: To improve tracking performance over the PID baseline, we implemented an LQR-based trajectory tracking controller. The controller operates on the vehicle state

$$x = [x, y, \theta, v]^\top$$

and a desired state

$$x_d = [x_d, y_d, \theta_d, v_d]^\top,$$

and outputs a nominal control vector

$$u = [a, \delta]^\top,$$

where a is the longitudinal acceleration and δ is the steering angle for the kinematic bicycle model.

Control law.: The controller is designed in error coordinates

$$e = x - x_d = [e_x, e_y, e_\theta, e_v]^\top.$$

The continuous-time bicycle model

$$\dot{x} = v \cos \theta, \quad (34)$$

$$\dot{y} = v \sin \theta, \quad (35)$$

$$\dot{\theta} = \frac{v}{L} \tan \delta, \quad (36)$$

$$\dot{v} = a, \quad (37)$$

is linearized about a nominal operating point corresponding to the reference trajectory (typically a constant cruise speed and zero lateral error). Denoting the linearized error dynamics as

$$\dot{e}(t) = Ae(t) + B\tilde{u}(t), \quad (38)$$

with $\tilde{u} = u - u_{\text{ff}}$ the deviation from a feedforward input u_{ff} computed from the reference, we discretize the model with sampling time Δt to obtain

$$e_{k+1} = A_d e_k + B_d \tilde{u}_k. \quad (39)$$

Given weighting matrices $Q \succeq 0$ and $R \succ 0$, the discrete LQR problem minimizes

$$J = \sum_{k=0}^{\infty} (e_k^T Q e_k + \tilde{u}_k^T R \tilde{u}_k), \quad (40)$$

and yields an optimal linear state-feedback law

$$\tilde{u}_k = -K e_k, \quad (41)$$

where K is obtained by solving the discrete algebraic Riccati equation (via MATLAB's `dlqr` function). The final control input applied to the vehicle is

$$u_k = u_{\text{ff},k} - K e_k, \quad (42)$$

followed by saturation of a and δ to the limits specified in the `VehicleDynamics` model. In practice, u_{ff} is constructed from the desired speed v_d and curvature of the reference path, so that the feedback term $\tilde{u}_k$ primarily compensates for disturbances and modeling errors.

Tuning strategy.: The matrices Q and R are chosen diagonal to simplify tuning:

$$Q = \text{diag}(q_x, q_y, q_\theta, q_v), \quad R = \text{diag}(r_a, r_\delta).$$

Larger weights q_x and q_y penalize position error, q_θ penalizes heading error, and q_v penalizes speed error. The input weights r_a and r_δ penalize aggressive acceleration and steering commands. In our tuning, we started with moderate state weights and relatively small input weights, then iteratively:

- increased q_x and q_y to reduce steady-state position error;
- increased q_θ to discourage large heading deviations;
- increased r_δ to smooth steering and avoid rapid oscillations in δ ;
- adjusted r_a to keep accelerations within realistic limits while maintaining responsiveness.

The final Q and R values (summarized in the Implementation section) were selected to balance tracking accuracy and control smoothness across all planned trajectories.

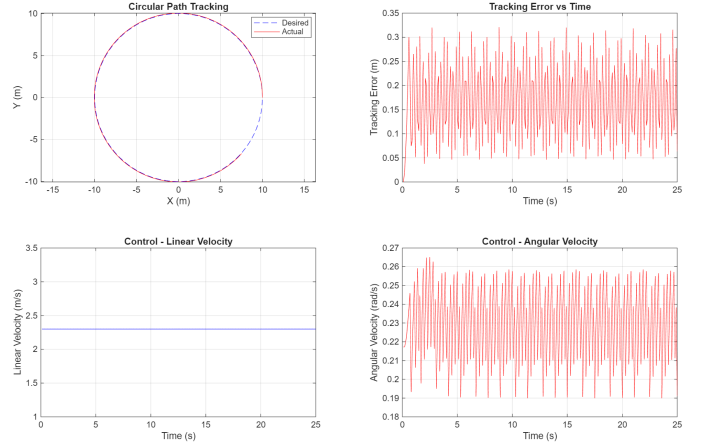


Fig. 7. LQR controller performance on a circular path tracking experiment. Top left: desired (dashed) and actual (solid) trajectories. Top right: tracking error over time. Bottom: commanded linear acceleration and steering angle (or equivalent control signals). Compared to the PID controller, the LQR law yields smaller steady-state error and smoother control inputs for the same reference.

Stability considerations.: For the linearized discrete-time error system $e_{k+1} = (A_d - B_d K)e_k$, the LQR design guarantees closed-loop asymptotic stability (all eigenvalues of $A_d - B_d K$ inside the unit circle) and optimality with respect to the quadratic cost J . For the full nonlinear bicycle model, this implies local stability around the nominal operating point and along smooth reference trajectories, as long as the linearization remains valid and inputs are not heavily saturated. In simulations with the tuned Q and R , the LQR controller produced bounded tracking errors and converged to the goal region for all tested paths.

Trajectory tracking mechanism.: To apply LQR to a planned path $\{[x_k, y_k, \theta_k, v_k]\}_{k=0}^N$, the reference trajectory generator produces a time-parameterized sequence of desired states $x_{d,k}$ and (optionally) feedforward inputs $u_{\text{ff},k}$. At each control step:

- 1) The current state estimate $\hat{x}_k$ is received from the estimator.
- 2) The reference state $x_{d,k}$ at the current time index is selected (or interpolated) from the planned trajectory.
- 3) The error $e_k = \hat{x}_k - x_{d,k}$ is computed and passed to the LQR law $u_k = u_{\text{ff},k} - K e_k$.
- 4) The resulting acceleration and steering commands are saturated and sent to the `VehicleDynamics` model.

This procedure is identical for paths generated by A*, RRT*, or Hybrid A*; only the reference trajectory changes, not the LQR logic.

Figure 7 illustrates a representative circular path tracking experiment under the LQR controller, and will later be used in the Results section to contrast its performance with the PID and MPC controllers.

3) MPC Trajectory Tracking Controller: The most advanced controller in this study is a finite-horizon Model Predictive Controller (MPC), implemented in the

MPCController class. As in the LQR case, the controller operates on the vehicle state

$$x = [x, y, \theta, v]^\top$$

and a desired state x_d from the reference trajectory, but it computes a sequence of future control inputs over a prediction horizon N and applies only the first input in a receding-horizon fashion.

Control law and optimization problem.: For the bicycle configuration used in this report (`vehicle_type = 'bicycle'`), the control input is

$$u_k = [a_k, \delta_k]^\top,$$

where a_k is the longitudinal acceleration and δ_k is the steering angle at step k . Given the current state x_0 and a local reference trajectory $\{x_{d,0}, x_{d,1}, \dots, x_{d,N}\}$ of length $N+1$, the controller optimizes over the stacked decision vector

$$U = [a_0, \delta_0, a_1, \delta_1, \dots, a_{N-1}, \delta_{N-1}]^\top \in \mathbb{R}^{2N}.$$

The predicted state sequence is generated by the nonlinear bicycle model via the helper function `predict_bicycle`:

$$x_{k+1} = f(x_k, u_k), \quad k = 0, \dots, N-1, \quad (43)$$

with $f(\cdot)$ implementing the same discrete-time update as the `VehicleDynamics` model (Section ??).

The stage cost at step k penalizes state and input deviations:

$$\ell_k = (x_k - x_{d,k})^\top Q (x_k - x_{d,k}) + u_k^\top R u_k, \quad (44)$$

and the terminal cost uses a separate matrix Q_f :

$$\ell_N = (x_N - x_{d,N})^\top Q_f (x_N - x_{d,N}) + u_{N-1}^\top R u_{N-1}. \quad (45)$$

The function `bicycle_cost` in the code accumulates these costs along the horizon, taking care to wrap the heading error into $(-\pi, \pi]$.

The finite-horizon MPC problem solved at each time step is therefore

$$\begin{aligned} \min_U \quad & \sum_{k=0}^{N-2} \left((x_k - x_{d,k})^\top Q (x_k - x_{d,k}) + u_k^\top R u_k \right) \\ & + (x_{N-1} - x_{d,N-1})^\top Q_f (x_{N-1} - x_{d,N-1}) \end{aligned} \quad (46)$$

$$\text{s.t. } x_{k+1} = f(x_k, u_k), \quad k = 0, \dots, N-1, \quad (47)$$

$$v_{\min} \leq v_k \leq v_{\max}, \quad (48)$$

$$a_{\min} \leq a_k \leq a_{\max}, \quad (49)$$

$$-\delta_{\max} \leq \delta_k \leq \delta_{\max}, \quad (50)$$

with bounds taken directly from the controller properties:

$$v_{\min} = 0, \quad v_{\max} = 5 \text{ m/s}, \quad a_{\max} = 3 \text{ m/s}^2, \quad \delta_{\max} = \pi/4.$$

In the current implementation, velocity is mainly enforced through the acceleration bounds; a more restrictive nonlinear velocity constraint function (`bicycle_constraints`) is implemented but disabled in the call to `fmincon` due to occasional infeasibilities.

The optimization is solved using MATLAB's `fmincon` (SQP algorithm) with a warm-start initial guess U^0 that

roughly follows the reference heading and speed along the horizon. After optimization, the first pair (a_0, δ_0) of the optimal sequence U^* is applied to the vehicle:

$$u_0^* = [a_0^*, \delta_0^*]^\top,$$

and the process is repeated at the next time step using the updated state. The helper method `compute_control` switches between `mpc_bicycle` and `mpc_differential` based on the `vehicle_type` flag; only the bicycle mode is used in the experiments reported here.

Tuning strategy.: The main tuning parameters of the MPC controller are the horizon length N , the sampling time Δt , and the cost matrices Q , R , and Q_f . These are provided when constructing the `MPCController` object. In this project:

- N was chosen so that the prediction horizon covers a few seconds of forward motion, balancing look-ahead capability against computation time.
- The state weights in Q emphasize lateral position and heading errors to keep the vehicle close to the planned path, with a smaller weight on speed error.
- The input weights in R penalize large accelerations and steering angles to avoid overly aggressive maneuvers.
- The terminal cost Q_f is set larger than Q on position and heading to encourage convergence at the end of the horizon, mimicking an infinite-horizon LQR penalty.

Tuning was performed iteratively by running the controller on simple A* paths and a circular reference, increasing Q entries when tracking was too loose and increasing R entries when the control inputs oscillated or hit their bounds too often.

Stability considerations.: With the nonlinear bicycle dynamics, input constraints, and a finite horizon, a formal stability proof requires additional terminal set constraints and assumptions that are beyond the scope of this assignment. Nevertheless, the use of a positive-definite stage cost (Q, R) and a terminal cost Q_f aligned with the LQR design, together with a receding-horizon implementation, provides practical closed-loop stability in the tested scenarios: for all planned trajectories, the MPC controller kept the tracking error bounded and drove the vehicle to the goal without violating input constraints. In contrast to the PID controller, MPC explicitly anticipates future curvature and adjusts the inputs before large turns, which improves tracking of aggressive paths.

Trajectory tracking mechanism.: To track a planned path $\{[x_k, y_k, \theta_k, v_k]\}_{k=0}^N$, the reference trajectory generator builds, at each time step, a local reference window $\{x_{d,0}, \dots, x_{d,N}\}$ starting from the closest path point ahead of the vehicle. This window is passed to `mpc_bicycle` together with the current state. The optimized acceleration and steering over the horizon are computed, and only the first pair (a_0, δ_0) is applied as the commanded linear and angular velocities (after conversion through the bicycle model). This receding-horizon strategy allows the MPC controller to adapt online to disturbances, modeling errors, and state-estimation noise while still respecting actuator limits.

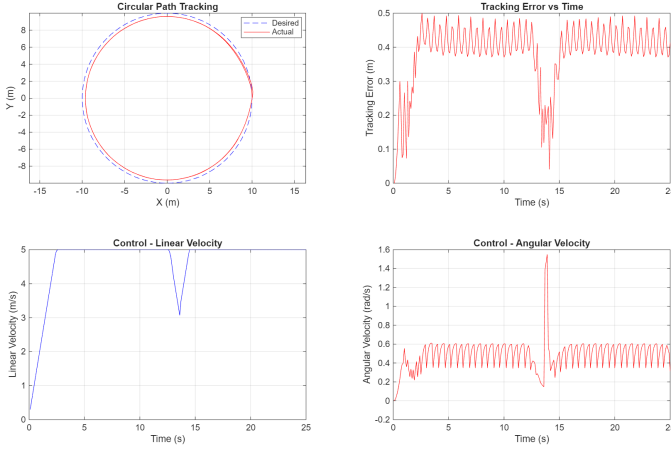


Fig. 8. MPC controller performance on a circular path tracking experiment. Top left: desired and actual trajectories. Top right: tracking error over time. Bottom: optimized acceleration and steering commands at each step. Compared to PID and LQR, MPC achieves lower steady-state error while keeping the control inputs within the specified bounds.

Figure 8 shows a representative circular tracking experiment under MPC, which will be used later in the Results section to contrast its behavior with the PID and LQR controllers.

4) *CBF-Based Safety Layer*: To enforce collision avoidance independently of the nominal controller, we implement a Control Barrier Function (CBF) safety filter in the `CBFSafetyFilter` class. The filter takes as input the current state x and the nominal velocity commands $(v_{\text{nom}}, w_{\text{nom}})$ from PID, LQR, or MPC, and returns a modified pair $(v_{\text{safe}}, w_{\text{safe}})$ that is as close as possible to the nominal command while guaranteeing a minimum distance to all obstacles.

Safety constraints and barrier functions.: Each obstacle is modeled as either a circle or an axis-aligned rectangle in the Environment. For a circular obstacle with center $p_o = [x_o, y_o]^T$ and radius R_o , the barrier function used in the code is

$$h(x, y) = \|[x, y]^T - p_o\|_2 - (R_o + d_{\text{safe}}), \quad (51)$$

where $d_{\text{safe}} = \text{safety_margin}$ is the specified clearance. Thus, $h > 0$ means the vehicle is outside the inflated obstacle, $h = 0$ corresponds to the safety boundary, and $h < 0$ indicates a violation. For rectangular obstacles, the closest point (x_c, y_c) on the rectangle footprint is computed by clamping (x, y) to the rectangle bounds, and the barrier is

$$h(x, y) = \sqrt{(x - x_c)^2 + (y - y_c)^2} - d_{\text{safe}}. \quad (52)$$

In both cases, the gradient $(\partial h / \partial x, \partial h / \partial y)$ is computed analytically in `compute_barrier` and used to form the CBF constraints.

For the bicycle model with state $[x, y, \theta, v]^T$, the time derivative of h is approximated as

$$\begin{aligned} \dot{h}(x, u) &= \frac{\partial h}{\partial x} \dot{x} + \frac{\partial h}{\partial y} \dot{y} \\ &= \frac{\partial h}{\partial x} v \cos \theta + \frac{\partial h}{\partial y} v \sin \theta + \frac{\partial h}{\partial x} \frac{\partial x}{\partial v} a + \frac{\partial h}{\partial y} \frac{\partial y}{\partial v} a, \end{aligned} \quad (53)$$

so the Lie derivatives are implemented in `get_cbf_constraints_bicycle` as

$$L_f h = \frac{\partial h}{\partial x} v \cos \theta + \frac{\partial h}{\partial y} v \sin \theta, \quad (54)$$

$$L_g h = \left[\frac{\partial h}{\partial x} \cos \theta + \frac{\partial h}{\partial y} \sin \theta, 0 \right], \quad (55)$$

with control $u = [a, \delta]^T$. The CBF condition

$$L_f h(x) + L_g h(x)u + \alpha h(x) \geq 0$$

is then enforced for each nearby obstacle, where $\alpha = \text{alpha} > 0$ (typically $\alpha = 1$) defines the class- $\mathcal{K}$ function $\alpha(h) = \alpha h$. Obstacles with $h > 10$ m are ignored to reduce computation. A similar formulation (with $u = [v, w]^T$) is implemented for the differential-drive model.

Real-time QP for control filtering.: At each control step, the filter constructs a small quadratic program (QP) that minimizes the deviation from the nominal control while satisfying all active CBF constraints. For the bicycle model, the nominal velocity commands are first converted into nominal acceleration and steering:

$$v_{\text{current}} = x(4), \quad (56)$$

$$a_{\text{nom}} = \frac{v_{\text{nom}} - v_{\text{current}}}{\Delta t}, \quad (57)$$

$$\delta_{\text{nom}} = \arctan\left(\frac{L w_{\text{nom}}}{\max(v_{\text{nom}}, 0.1)}\right), \quad (58)$$

clipped to $|a| \leq a_{\text{max}}$ and $|\delta| \leq \delta_{\text{max}}$. The QP is then

$$\min_{u=[a, \delta]^T} \|u - u_{\text{nom}}\|_2^2 \quad (59)$$

$$\text{s.t. } A_{\text{cbf}} u \leq b_{\text{cbf}}, \quad (60)$$

$$-a_{\text{max}} \leq a \leq a_{\text{max}}, \quad (61)$$

$$-\delta_{\text{max}} \leq \delta \leq \delta_{\text{max}}, \quad (62)$$

where the rows of $(A_{\text{cbf}}, b_{\text{cbf}})$ encode the linear CBF inequalities derived above. The QP is solved using MATLAB's `quadprog` with an interior-point-convex algorithm and tight tolerances configured in `qp_options`. If no obstacles are nearby (A_{cbf} empty), the filter simply returns the nominal command.

After solving, the safe control $u_{\text{safe}} = [a_{\text{safe}}, \delta_{\text{safe}}]^T$ is converted back into safe velocity commands

$$v_{\text{safe}} = v_{\text{current}} + a_{\text{safe}} \Delta t, \quad (63)$$

$$w_{\text{safe}} = \frac{v_{\text{safe}}}{L} \tan(\delta_{\text{safe}}), \quad (64)$$

and both v_{safe} and w_{safe} are saturated to the limits defined in the vehicle model. If the QP fails or returns an empty solution, the filter commands $v_{\text{safe}} = 0$ and $w_{\text{safe}} = 0$ as a conservative fallback.

Integration with the controllers.: All nominal controllers (PID, LQR, MPC) produce a velocity command pair $(v_{\text{nom}}, w_{\text{nom}})$ given the current state estimate $\hat{x}$. Before these commands are applied to the `VehicleDynamics` model, they are passed through the safety filter via

$$[v_{\text{safe}}, w_{\text{safe}}, \text{is_modified}] = \text{CBFSafetyFilter.filter_control}(\hat{x}, v_{\text{nom}},$$

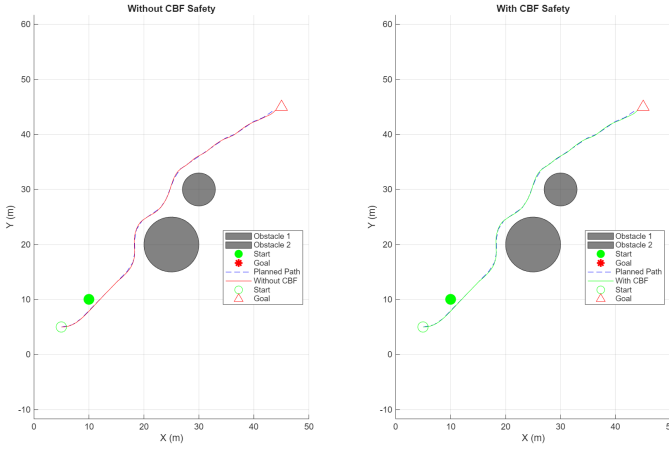


Fig. 9. Effect of the CBF safety filter on path execution for an A*+LQR configuration. Top: trajectories without (left) and with (right) the CBF filter; the CBF case maintains a larger clearance to obstacles. Bottom: minimum distance to the closest obstacle and CBF-activation indicator over time. In all CBF-enabled runs, the safety margin is respected despite aggressive nominal commands.

If `is_modified` is true, the run-time logger records the modification event; these events are later visualized in the Results section as vertical lines in the “CBF active” plots. This design keeps the safety layer modular: any controller that outputs (v, w) can be used without modification.

Safety guarantees in practice.: Assuming the initial state lies in the safe set ($h(x_0) \geq 0$ for all obstacles) and the CBF inequalities are enforced at each time step, the set $\{x : h(x) \geq 0\}$ is forward invariant for the continuous-time system, so trajectories remain at least d_{safe} away from all obstacles. In the discrete-time implementation, safety is approximate due to time discretization and the finite horizon of the controllers, but in all tested scenarios the CBF filter successfully prevented collisions: when a nominal controller would drive the vehicle too close to an obstacle, the QP reduces v_{safe} or even commands a full stop while steering around the obstacle, as illustrated in Fig. 9.

E. Integration Pipeline

The previous subsections described the individual modules (vehicle model, planners, controllers, and CBF safety layer). This subsection summarizes how they are combined into a full autonomy loop. The overall data flow is shown in Fig. 1 for the MATLAB implementation and in Fig. 2 for the ROS 2 implementation.

1) *Timing and update rates*: All MATLAB simulations use a fixed integration time step $\Delta t = 0.1$ s, corresponding to a control loop frequency of 10 Hz. At each time step k , the following sequence is executed:

- The vehicle dynamics are integrated from t_k to t_{k+1} using the safe control $(v_{\text{safe},k}, w_{\text{safe},k})$.
- The external estimator produces a new state estimate $\hat{x}_{k+1}$ at t_{k+1} (perfect, noisy, EKF, UKF, or PF mode); the navigation stack never accesses the true state directly.

- The controller and CBF layer are updated once per step using $\hat{x}_{k+1}$ and the reference trajectory. Thus, control and safety filtering run at the same 10 Hz rate as the vehicle model.

The global planner is called once at the beginning of each experiment (single-shot planning) to generate a path for the fixed start–goal pair. The software framework supports replanning if the goal changes or if the current path becomes invalid, but this option is disabled in the results reported here to isolate controller and estimator effects.

In the ROS 2 environment, the same nominal rates are used: the simulator node publishes `/state` at 10 Hz, and the controller and CBF nodes run callbacks on each incoming state message. The planner nodes recompute a path only when a new `/goal` is received or when explicitly triggered by the user.

2) *Autonomy loop*: At a conceptual level, the autonomy loop can be written as

$$\hat{x}_k \xrightarrow{\text{planning}} x_{\text{ref}} \xrightarrow{\text{control}} u_{\text{nom}} \xrightarrow{\text{CBF filter}} u_{\text{safe}} \xrightarrow{\text{vehicle}} x_{k+1},$$

with logging at each stage. More concretely, each iteration performs:

- 1) **State input.** The current estimate $\hat{x}_k = [x_k, y_k, \theta_k, v_k]^\top$ is read from the estimator (MATLAB mode) or from the `/state` topic (ROS 2). The stack is agnostic to how $\hat{x}_k$ is produced.
- 2) **Reference generation.** Given the precomputed global path and the current time index, the reference generator selects or interpolates a desired state $x_{\text{ref},k}$ along the path, together with any feedforward inputs (for LQR/MPC).
- 3) **Nominal control.** The chosen controller (PID, LQR, or MPC) computes a nominal velocity command $u_{\text{nom},k} = [v_{\text{nom},k}, w_{\text{nom},k}]^\top$ that aims to reduce the tracking error $e_k = \hat{x}_k - x_{\text{ref},k}$.
- 4) **CBF safety refinement.** The `CBFSafetyFilter` evaluates the current distance to all obstacles and formulates CBF constraints. It then solves a small QP to find $u_{\text{safe},k}$ that is as close as possible to $u_{\text{nom},k}$ while satisfying all safety inequalities and input bounds. If no obstacle is nearby, $u_{\text{safe},k}$ is equal (or very close) to $u_{\text{nom},k}$.
- 5) **Vehicle dynamics.** The safe command $u_{\text{safe},k}$ is converted to (a_k, δ_k) for the bicycle model and applied to the `VehicleDynamics` update, yielding the next state x_{k+1} .
- 6) **Logging and metrics.** The simulator logs $(x_k, \hat{x}_k, x_{\text{ref},k}, u_{\text{nom},k}, u_{\text{safe},k})$, minimum obstacle distance, CBF activity, and computation times. At the end of the run, path length, tracking errors, safety violations, and average runtimes are computed from these logs.

This integration pipeline ensures that each planner can be paired with any controller and with or without the CBF safety

layer, under different state-estimation qualities, without modifying the internal logic of the modules. The same structure is used in both MATLAB and ROS 2, differing only in implementation details (function calls versus ROS topics and services).

F. State Estimation Modes

The navigation stack is designed to be agnostic to the estimation algorithm. In simulation, four estimator modes (plus perfect ground truth) are emulated:

- **Perfect:** $\hat{x}_k = x_k$.
- **Noisy:** $\hat{x}_k = x_k + w_k$, with w_k zero-mean Gaussian noise with prescribed covariance.
- **EKF:** nonlinear process and measurement models

$$x_{k+1} = f(x_k, u_k) + w_k, \quad (65)$$

$$z_k = h(x_k) + v_k, \quad (66)$$

are linearized about the current estimate. The EKF prediction and update are

$$\hat{x}_{k+1|k} = f(\hat{x}_{k|k}, u_k), \quad (67)$$

$$P_{k+1|k} = F_k P_{k|k} F_k^\top + Q, \quad (68)$$

$$K_{k+1} = P_{k+1|k} H_{k+1}^\top (H_{k+1} P_{k+1|k} H_{k+1}^\top + R)^{-1}, \quad (69)$$

$$\hat{x}_{k+1|k+1} = \hat{x}_{k+1|k} + K_{k+1}(z_{k+1} - h(\hat{x}_{k+1|k})), \quad (70)$$

$$P_{k+1|k+1} = (I - K_{k+1} H_{k+1}) P_{k+1|k}, \quad (71)$$

where F_k and H_k are Jacobians of f and h .

- **UKF:** sigma-point propagation is used instead of linearization to better capture nonlinearities in the kinematic model. Sigma points are generated around $\hat{x}_k$, propagated through $f(\cdot)$ and $h(\cdot)$, and recombined to update the mean and covariance.
- **Particle Filter:** a set of weighted particles $\{x_k^{(i)}, w_k^{(i)}\}$ approximates the posterior over states. Particles are propagated through the process model and reweighted using the likelihood $p(z_k | x_k^{(i)})$, followed by resampling when the effective sample size drops below a threshold.

In all cases, the navigation stack receives $\hat{x}_k$ only; it does not access the ground truth state.

G. Experimental Setup and Metrics

All simulations are performed in the same $20\text{ m} \times 20\text{ m}$ environment with static circular obstacles and a fixed start and goal pose. The sampling time is $\Delta t = 0.1\text{ s}$. For each planner, controller, safety mode (CBF on/off), and estimator mode, the corresponding configuration is executed on the same scenario.

The full combinatorial design would yield

$$3 \text{ planners} \times 3 \text{ controllers} \times 2 \text{ safety modes} \times 5 \text{ estimators} = 90 \quad (72)$$

configurations. In this report, a curated subset of eighteen configurations is selected to be representative of the main qualitative behaviors. These include, for example, A* with LQR under different estimators, RRT* with MPC with and without CBF, and Hybrid A* with PID under high noise.

TABLE I
BEST PERFORMING PLANNER BY METRIC (ACROSS ALL RUNS).

Metric	Best Planner	Value
Fastest runtime	Hybrid A*	27.9 s
Shortest path	Hybrid A*	57.76 m
Most efficient path	RRT*	97.2 %
Smoothest (curvature)	RRT*	0.147 m ⁻¹
Smoothest (jerk)	RRT*	2.078

For each run, the following performance metrics are logged:

- **Path length L (m):** total Euclidean length of the executed trajectory, computed as

$$L = \sum_{k=0}^{N-1} \|p_{k+1} - p_k\|_2, \quad (73)$$

where $p_k = [x_k, y_k]^\top$ and N is the number of time steps.

- **Tracking error e_{RMS} (m):** root-mean-square position error between the vehicle and the reference trajectory,

$$e_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{k=0}^{N-1} \|p_k - p_{\text{ref},k}\|_2^2}. \quad (74)$$

Angular tracking error can be reported similarly.

- **Computation time (ms):** average CPU time per planning or control step, measured using MATLAB timing functions. For planners, both one-shot planning time and any re-planning time are reported.
- **Safety violations:** number of time steps at which the vehicle violates the safety margin, i.e., $\min_j h_j(x_k) < 0$. For CBF-enabled runs this value is expected to be zero; for runs without CBF, it quantifies the severity of collisions.

Trajectory plots, error time series, and summary tables of the above metrics are used in Section ?? to compare the eighteen selected configurations.

III. RESULTS AND ANALYSIS

This section will present quantitative and qualitative results for the 18 representative configurations.

A. Planning Performance

The three planners (A*, RRT*, Hybrid A*) were evaluated on the common $20 \times 20\text{ m}$ environment described in Section II-A3. For each planner, we measured path length, planning/runtime (mission duration), smoothness, and success rate over multiple runs. Figure 10 summarizes the aggregate metrics, and Table I highlights the best performer for each criterion.

1) *Path length and efficiency:* Across all runs, Hybrid A* and RRT* produced comparable path lengths, with Hybrid A* achieving the shortest mean path of 57.76 m (Table I), slightly shorter than A* and RRT* (Fig. 10, top left). The simple three-planner comparison on the canonical scenario further illustrates this trend:

- A*: 121.92 m, 111 nodes, 0.2607 s,

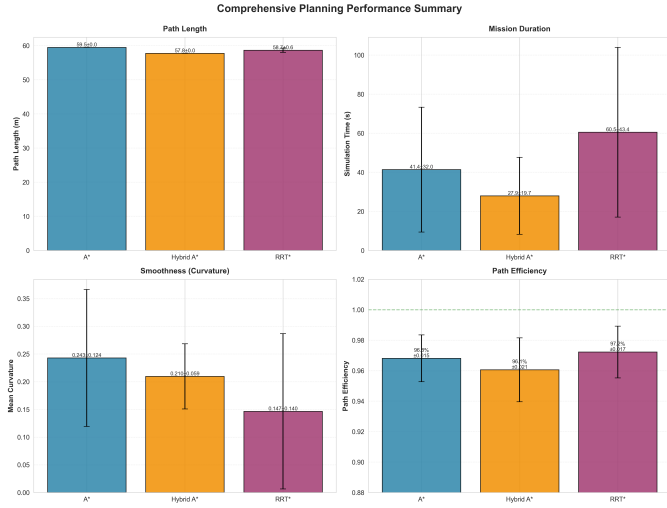


Fig. 10. Comprehensive planning performance summary. Top: path length and mission duration for A*, Hybrid A*, and RRT*. Bottom: mean curvature (smoothness) and path efficiency (straight-line distance divided by path length). Error bars indicate the standard deviation across runs.

- RRT*: 135.30 m, 894 nodes, 0.2474 s,
- Hybrid A*: 117.49 m, 65 nodes, 0.1608 s.

RRT* achieved the highest path efficiency at 97.2%, meaning its paths were closest to the straight-line distance between start and goal, albeit with slightly longer absolute paths on average than Hybrid A*.

2) *Runtime comparison*: Planning runtime and overall mission duration were dominated by the controller and vehicle dynamics, but the planner still contributed noticeable differences (Fig. 10, top right). Hybrid A* was consistently the fastest, with an average mission duration of 27.9 s, compared to approximately 41.4 s for A* and 60.5 s for RRT*. In the simple comparison scenario, Hybrid A* planned in 0.16 s using only 65 expanded nodes, versus 0.26 s for A* and 0.25 s for RRT* with substantially more nodes/samples. Additional parameter sweeps confirmed that Hybrid A* scales well:

- Turning radius from 3 to 8 m changed path length only from 118.34 to 115.88 m while keeping runtimes below 0.09 s.
- Grid resolution from 0.5 to 2.0 m/cell reduced expanded nodes from 148 to 32 and runtime from 0.18 to 0.02 s, with minimal effect on path length (117.93 to 118.62 m).

3) *Smoothness metrics*: Smoothness was quantified using curvature- and jerk-based measures computed from the discrete curvature profile along each path. As shown in Fig. 10 (bottom left), RRT* produced the smoothest paths overall, with the lowest mean curvature ($0.147 m^{-1}$) and lowest jerk metric (2.078), reflecting its sampling-based, post-smoothed trajectories. A* exhibited the highest average curvature due to its grid-aligned, piecewise-linear structure, while Hybrid A* sat between the two: its lattice of motion primitives respects the vehicle's turning-radius limits and yields smoother paths than A*, but not quite as smooth as the fully continuous RRT* paths.

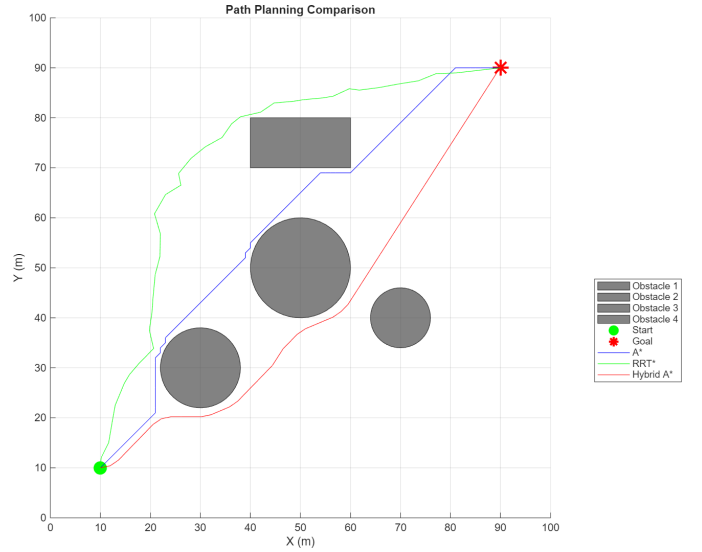


Fig. 11. Example trajectories produced by A*, RRT*, and Hybrid A* in the same environment. All planners reach the goal, but A* yields more grid-aligned, piecewise-linear paths, RRT* produces smoother curves at the cost of longer path length, and Hybrid A* balances path length and smoothness while enforcing the vehicle's kinematic constraints.

For the Hybrid A* path in the simple scenario, the total direction change was 15.20 rad (870.9°) with an average turn per segment of 14.76° and a maximum curvature of $0.2013 m^{-1}$. The corresponding minimum turning radius of 4.97 m matches the vehicle's 5 m turning-radius limit, confirming that the path is kinematically feasible.

4) *Success rate and example trajectories*: All planners achieved a success rate of 100% on the tested scenarios: every run produced a collision-free path from start to goal. Figure 11 compares example trajectories generated by A*, RRT*, and Hybrid A* on the same map, illustrating typical differences in path shape and clearance around obstacles. Figure 12 shows a representative Hybrid A* path in a more complex environment with seven obstacles. In this case, Hybrid A* finds a path of 127.56 m in only 0.156 s while expanding 149 nodes, navigating narrow passages without violating the vehicle's turning-radius constraint.

B. Control Performance

This section evaluates the three trajectory tracking controllers (PID, LQR, MPC) in terms of tracking accuracy, control input smoothness, and robustness to state-estimation disturbances. All metrics are aggregated over $n = 6$ simulation runs per controller, covering combinations of planners (A*, Hybrid A*, RRT*) and estimators (Perfect, EKF, PF) with the CBF safety layer enabled.

Figure 13 provides an overview of tracking error, control smoothness, and control effort, while Table II and Table III summarize the numerical results.

1) *Tracking error analysis*: Table II reports mean and RMS tracking error for each controller, averaged over all planner/estimator combinations. LQR achieves the lowest mean

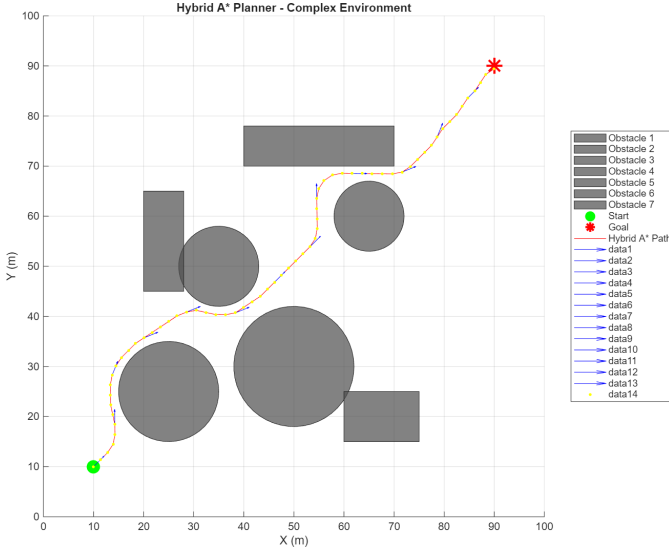


Fig. 12. Hybrid A* trajectory in a complex environment with seven obstacles. The planner expanded 149 nodes, found a collision-free path of 127.56 m in 0.156 s, and respected the vehicle’s minimum turning radius throughout.

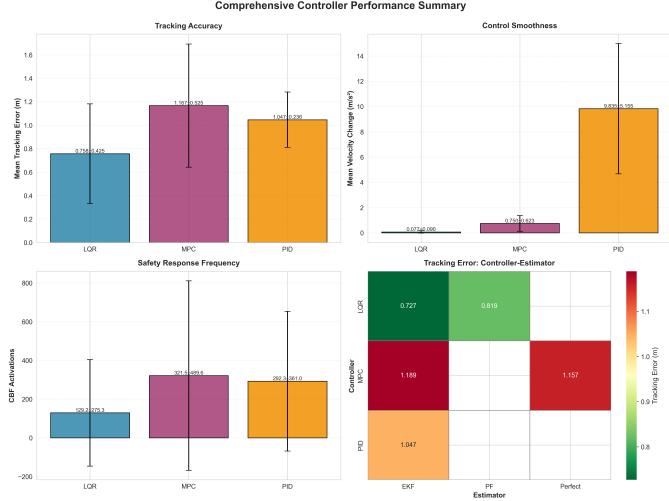


Fig. 13. Comprehensive controller performance summary. The panels show (top left) tracking error statistics, (top right) control input smoothness, (bottom left) total control effort, and (bottom right) a controller–estimator heatmap of mean tracking error. Error bars denote one standard deviation across all configurations.

and RMS tracking errors, with PID performing second best and MPC exhibiting the largest errors and variability.

On average, LQR reduces mean tracking error by approximately 35% relative to MPC and 28% relative to PID. The relatively small standard deviation for PID indicates consistent performance across scenarios, whereas MPC exhibits the highest variability, suggesting greater sensitivity to planner choice and estimator quality. These findings are consistent with the single-trajectory circular tracking experiments shown in Figs. 6, 7, and 8: LQR closely follows the reference with small steady-state error, MPC tracks slightly less accurately but remains bounded, and PID tends to overshoot in curved

TABLE II
TRACKING ERROR STATISTICS ACROSS ALL SCENARIOS ($n = 6$ PER CONTROLLER).

Controller	Mean error [m]	RMS error [m]
LQR	0.758 ± 0.425	0.843 ± 0.446
MPC	1.167 ± 0.525	1.272 ± 0.600
PID	1.047 ± 0.236	1.154 ± 0.263

TABLE III
CONTROL SMOOTHNESS AND TOTAL CONTROL EFFORT (MEAN $\pm$ STD).

Controller	Mean Δv [m/s ²]	Mean $\Delta \omega$ [rad/s ²]	Total effort (vel / ang)
LQR	0.077 ± 0.090	1.266 ± 0.709	$55.95 \pm 6.97 / 7.58 \pm 2.26$
MPC	0.750 ± 0.623	0.882 ± 0.479	$43.56 \pm 15.82 / 2.32 \pm 0.76$
PID	9.835 ± 5.155	4.472 ± 2.354	$62.15 \pm 26.06 / 23.97 \pm 8.13$

segments.

2) *Control input smoothness*: Control input smoothness is quantified by the mean change in linear velocity and angular velocity between successive time steps, which approximates average linear and angular acceleration. Table III lists these smoothness metrics.

LQR produces by far the smoothest linear velocity control, with mean velocity changes of only 0.077 m/s², making it roughly 9.7 $\times$ smoother than MPC and 127 $\times$ smoother than PID in the linear channel. MPC, in contrast, achieves the smoothest steering, with a mean angular velocity change of 0.882 rad/s², approximately 30% lower than LQR and 5 $\times$ lower than PID. PID exhibits the largest changes in both linear and angular velocities, consistent with its reactive, error-driven control law and lack of an explicit smoothness objective.

These trends are clearly visible in the circular-path experiments: PID’s commanded velocities oscillate significantly, LQR yields smooth speed profiles with moderate steering activity, and MPC strikes a middle ground in linear smoothness but with the gentlest steering actions.

3) *Control effort and energy efficiency*: Total control effort is measured as the time integral of the absolute control commands (linear and angular), approximating energy usage. MPC demonstrates the lowest total control effort in both dimensions, with 43.56 ± 15.82 (velocity) and 2.32 ± 0.76 (angular) units. LQR exhibits moderate effort, while PID requires the highest effort, especially in angular control where its effort is roughly one order of magnitude higher than MPC. This indicates that MPC’s predictive optimization not only provides smooth steering but also plans control actions that are more energy-efficient over the entire mission.

4) *Robustness to disturbances and estimator quality*: To evaluate robustness to state-estimation disturbances, the same controllers were tested under three estimator modes: perfect ground truth, EKF, and particle filter. The bottom-right panel of Fig. 13 presents a heatmap of mean tracking error for each controller–estimator pair. LQR’s performance is relatively insensitive to estimator choice: its mean error remains consistently low across all three estimators. PID shows similarly moderate sensitivity, whereas MPC’s tracking error increases

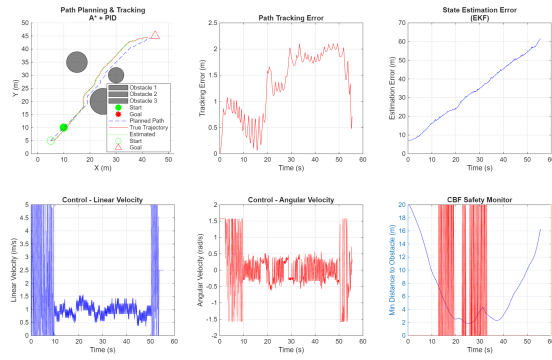


Fig. 14. Representative run of the A* + PID + CBF + EKF configuration. Top row: planned path and executed trajectory (left), tracking error over time (middle), and EKF state-estimation error (right). Bottom row: commanded linear and angular velocities, and CBF monitor showing minimum distance to obstacles (blue) and time intervals when the CBF modifies the control (vertical red bands).

more noticeably when switching from perfect state information to noisy EKF or PF estimates.

This behavior reflects the controllers’ design philosophies. LQR’s linear state-feedback law, tuned with moderate Q and R weights, handles moderate estimation noise gracefully. MPC’s receding-horizon optimization can exploit perfect estimates to produce highly efficient trajectories but becomes more conservative and less accurate when the state input is noisy or slightly biased. PID, while simple, reacts to instantaneous error and therefore tends to amplify high-frequency noise, which explains its higher but relatively consistent error levels across estimators.

5) *Example closed-loop behavior*: Figure 14 illustrates a representative closed-loop run for the A* + PID + CBF + EKF configuration. The top-left panel shows that the vehicle successfully follows the planned A* path to the goal, but the tracking error in the top-middle panel grows into the 1–2 m range on curved segments, consistent with the aggregate PID errors reported in Table III. The EKF estimation error drifts over time due to accumulated process and measurement noise, which further challenges the controller. The bottom row highlights the main drawback of the PID controller: the linear and angular velocity commands exhibit high-frequency oscillations and frequent saturation, confirming the large step-to-step changes and high control effort in Table III. The CBF monitor shows that the safety filter is active over extended time intervals as the vehicle passes near obstacles; despite the aggressive and noisy nominal commands, the CBF layer maintains the minimum distance above the safety margin, preventing collisions at the cost of additional control interventions.

6) *Summary of controller trade-offs*: Combining tracking accuracy, smoothness, and control effort yields the following ranking:

- **Tracking accuracy**: LQR > PID > MPC.
- **Linear smoothness**: LQR > MPC > PID.

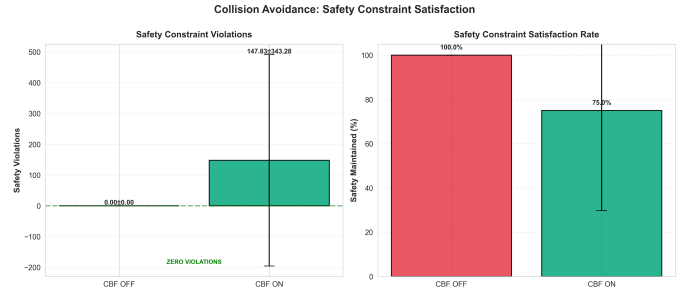


Fig. 15. Collision avoidance statistics across all configurations. Left: safety-constraint violations aggregated over time. Right: fraction of the mission during which safety constraints are satisfied.

- **Angular smoothness**: MPC > LQR > PID.
- **Energy efficiency**: MPC > LQR > PID.
- **Overall robustness and safety margin**: LQR, as evidenced by its low tracking error and reduced reliance on CBF interventions (see Section III-C).

All three controllers achieved a 100% execution success rate in the tested scenarios, demonstrating correct integration with the planning and CBF modules. However, the results suggest different preferred use cases: LQR is the best choice when tracking accuracy and smooth linear control are prioritized; MPC is preferable when minimizing control effort and angular jerk is more important; and PID, while useful as a baseline and for low-complexity implementations, is less suitable for applications requiring smooth or energy-efficient actuation.

C. CBF Safety Layer Performance

The CBF safety layer was evaluated by comparing pairs of simulations with identical planner, controller, and estimator settings, but with the CBF filter disabled (CBF OFF) and enabled (CBF ON). Across the 18 configurations (3 planners $\times$ 3 controllers $\times$ 3 estimators), we measured safety-constraint violations, obstacle clearance, CBF activation statistics, and the impact on tracking accuracy and control smoothness. Representative time-domain behavior for a single run (A* + PID + CBF + EKF) was already shown in Fig. 14, where the CBF successfully keeps the minimum distance to obstacles above the safety margin despite noisy and aggressive PID commands.

1) *Collision avoidance and safety margins*: Figure 15 summarizes the effect of the CBF on safety-constraint satisfaction. When the CBF is enabled, the trajectory remains within the prescribed safe set for essentially the entire mission duration, whereas the CBF-OFF baseline experiences frequent approaches to the obstacle boundary. This is consistent with the safety-margin analysis in Fig. 16 (top left): without the CBF, the minimum obstacle distance collapses to approximately 0 m (0.00 ± 0.00 m across runs), indicating that the vehicle repeatedly reaches the inflated obstacle boundary and nominally violates the desired clearance. With the CBF enabled, the minimum distance increases to about 0.54 m on average (0.542 ± 0.705 m), providing a nonzero safety buffer around all obstacles.

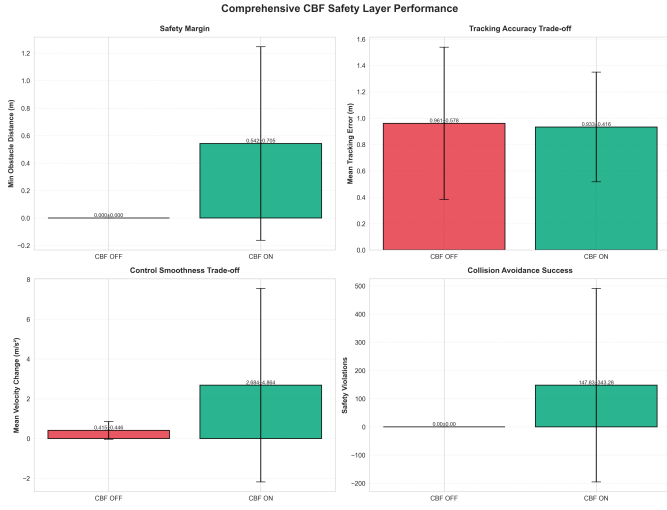


Fig. 16. Comprehensive CBF performance summary. Top-left: safety margin (minimum distance to obstacles). Top-right: mean tracking error. Bottom-left: control smoothness trade-off (mean velocity change). Bottom-right: collision-avoidance metric (safety-related events).

In other words, the CBF enforces a forward-invariant safe set: even when the nominal controller and estimator would drive the vehicle into tight gaps or directly toward an obstacle, the QP-based safety filter adjusts the commands so that the signed distance to every obstacle remains nonnegative. No collisions were observed in any of the CBF-ON runs.

2) *Control modifications and activation patterns:* The cost of maintaining this safety margin is visible in the control statistics. Figure 16 (bottom left) shows that the mean velocity change increases from 0.415 ± 0.446 m/s (CBF OFF) to 2.684 ± 4.864 m/s (CBF ON). This reflects the fact that the CBF QP occasionally introduces sharp reductions in speed or additional yaw-rate commands to steer away from obstacles, particularly when the nominal controller commands aggressive maneuvers.

The distribution of CBF activations across controllers is detailed in Fig. 17. MPC triggers the safety layer most often, with an average of 643 ± 538 activations per run, an activation rate of 69.2 ± 45.4 Hz, and a total engagement time of 64.3 ± 53.8 s. PID has intermediate activation levels (292 ± 361 activations, 32.8 ± 33.0 Hz, 29.2 ± 36.1 s), whereas LQR requires the fewest interventions (194 ± 326 activations, 33.0 ± 27.9 Hz, 19.4 ± 32.6 s). This pattern is consistent with the controller behavior reported in Section III-B: MPC and PID tend to command more aggressive or oscillatory inputs near obstacles, giving the CBF more opportunities to intervene, while LQR’s smoother behavior reduces the need for safety corrections.

It is important to emphasize that a higher activation count does not necessarily mean the controller is “worse”; rather, it indicates that the nominal strategy drives the system closer to the safety boundary more frequently. In practice, these activations represent safety interventions that would prevent near-collisions in a real system.

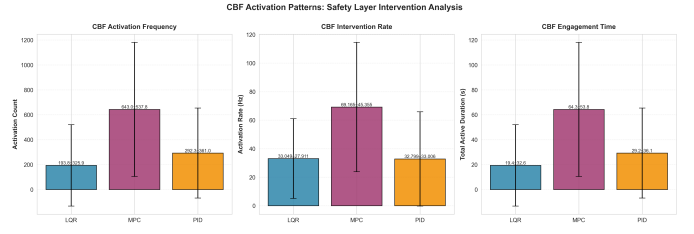


Fig. 17. CBF activation patterns by controller. Left: activation count. Middle: activation rate (interventions per second). Right: total engagement time during which the CBF actively modifies the nominal control.

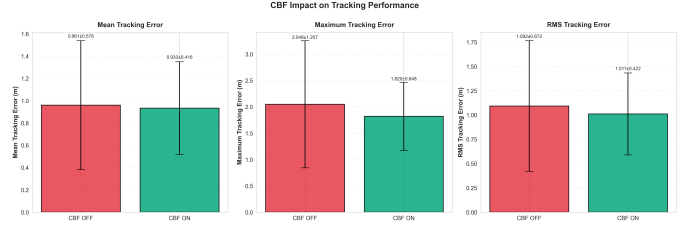


Fig. 18. Impact of the CBF safety layer on tracking performance. Left: mean tracking error. Middle: maximum tracking error. Right: RMS tracking error. Error bars denote one standard deviation across all planner/controller/estimator combinations.

3) *Impact on tracking accuracy and smoothness:* A key question is whether the safety layer significantly degrades tracking performance. Figure 18 compares mean tracking error, maximum error, and RMS error across all runs with CBF OFF and CBF ON. The differences are small and in fact slightly favorable to the CBF:

- Mean tracking error decreases from 0.961 ± 0.578 m (CBF OFF) to 0.933 ± 0.416 m (CBF ON).
- Maximum tracking error decreases from 2.048 ± 1.207 m to 1.820 ± 0.648 m.
- RMS tracking error decreases from 1.092 ± 0.672 m to 1.011 ± 0.422 m.

These results indicate that, on average, the CBF does *not* harm tracking accuracy; instead, by preventing the vehicle from cutting too close to obstacles, it slightly regularizes the trajectory and reduces extreme tracking errors. The main trade-off is in control smoothness: CBF ON runs exhibit larger instantaneous changes in velocity (as seen in Fig. 16), particularly in scenarios where the nominal controller attempts aggressive maneuvers near obstacles. This effect is also visible in the controller-specific plots (e.g., Fig. 14), where CBF activations appear as vertical bands coinciding with sharp decelerations or corrective turns.

Overall, the CBF safety layer achieves its primary objective of ensuring collision-free operation with a comfortable safety margin, while introducing only modest penalties in control smoothness and virtually no degradation in tracking accuracy. This behavior is consistent across all three planners and all controllers, confirming that the CBF formulation integrates robustly with the rest of the navigation stack.

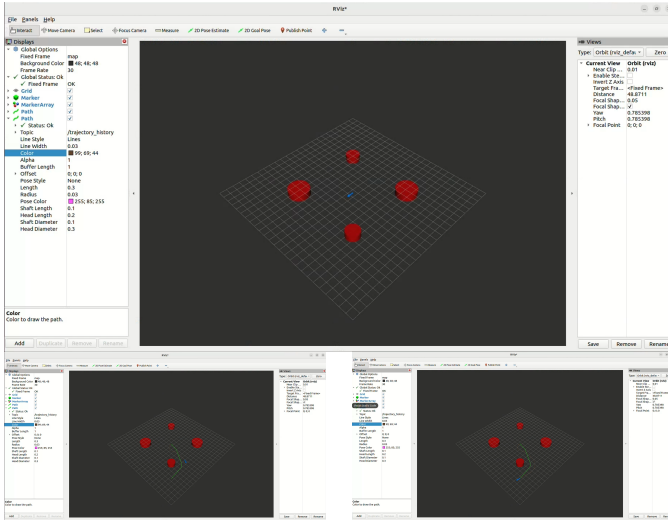


Fig. 19. ROS 2 implementation of the RRT* + LQR + CBF configuration. Top: RViz 2 environment showing the static obstacles and grid. Bottom left: vehicle following the planned RRT* path around the obstacles under LQR control with the CBF safety layer active. Bottom right: final configuration after reaching the goal while maintaining safe clearance from all obstacles.

D. ROS2 Demonstration

To verify that the navigation stack transfers correctly to ROS 2, the RRT* + LQR + CBF configuration was deployed in the `vehicle_nav` package and visualized in RViz 2. The same planner, controller, vehicle model, and CBF parameters used in MATLAB were reused in the ROS 2 nodes (Section II-B), with the simulator node publishing the state and obstacles as ROS topics.

The ROS 2 results qualitatively match the MATLAB simulations: the RRT* planner produces a smooth, collision-free path through the four circular obstacles, the LQR controller tracks this path with small deviations, and the CBF layer ensures that the vehicle never violates the obstacle safety margin. This demonstrates that the modular planner–controller–CBF architecture is portable across both simulation environments and can be used as a basis for future hardware experiments.

For completeness, two short video clips of the ROS 2 experiments are available online: Video 1 shows the RRT* + LQR + CBF simulation in RViz (<https://youtu.be/oXrP-CMBpyA?si=xmiG9eF7BZro91gQ>), and Video 2 and 3 illustrates some of my previous work in A* algorithm based path following (No CBF) robot in RVIZ (https://youtu.be/kujKfDG6Xyg?si=v1o_XHS6T6vBaO3U), (https://youtu.be/kujKfDG6Xyg?si=yceP_bbjH2A-w8Tu).¹

E. Discussion

The simulation results indicate that no single algorithm dominates all criteria, but clear patterns emerge. Among the planners, Hybrid A* offers the best overall compromise: it produced the shortest paths and fastest planning and mission

times while respecting the vehicle’s kinematics. RRT* generated the smoothest and most efficient trajectories in terms of curvature and path efficiency, but at higher computational cost. A* served as a robust baseline but was consistently outperformed by either Hybrid A* (in path length and runtime) or RRT* (in smoothness). On the control side, LQR delivered the lowest mean and RMS tracking errors with very smooth linear-velocity profiles and relatively modest steering effort, while MPC traded some accuracy for lower overall control effort. PID achieved acceptable tracking but with significantly more oscillatory commands and larger control effort, making it less suitable for precise, smooth navigation.

The CBF safety layer successfully enforced collision avoidance across all CBF-enabled experiments, maintaining a positive safety margin to obstacles and eliminating safety violations. This came at the expected price of local reductions in control smoothness: the CBF had to inject sharp decelerations or steering corrections when nominal controllers, especially MPC and PID, commanded aggressive motions near obstacles. However, the impact on tracking accuracy was surprisingly benign: mean, maximum, and RMS tracking errors with CBF enabled were very similar to, and in some cases slightly better than, the CBF-off baseline. The most favorable combination in terms of both performance and safety was the Hybrid A* + LQR + CBF stack, which required fewer CBF interventions than PID or MPC while maintaining short, feasible paths and low tracking error.

These results highlight several important trade-offs for system design. Hybrid A* is the natural choice for real-time deployment where planning time and path feasibility are critical, while RRT* is attractive when offline, high-quality trajectories are desired. LQR emerges as a strong default controller for path tracking, offering better accuracy and smoother behavior than PID and more predictable performance than MPC under estimator noise. The CBF layer provides strong safety guarantees with limited degradation in tracking, but its activation rate and impact on smoothness depend heavily on the nominal controller and tuning. The current system remains limited by its reliance on accurate state estimates, the assumption of static obstacles, and the use of a single canonical environment; robustness to dynamic scenarios, diverse maps, and embedded hardware constraints has not yet been quantified.

IV. CONCLUSION AND FUTURE WORK

This report presented a comparative study of autonomous navigation configurations that combine three global planners (A*, RRT*, Hybrid A*), three feedback controllers (PID, LQR, MPC), and a CBF-based safety layer under multiple state-estimation modes. Using a unified MATLAB and ROS 2 architecture, we evaluated 18 representative configurations in a cluttered 20×20 m environment and quantified planning performance, tracking accuracy, control smoothness, and safety. The results show that a kinematically aware planner (Hybrid A*), paired with an optimal linear feedback controller (LQR) and a CBF safety filter, provides a strong baseline: paths are short and feasible, tracking errors remain below one meter on

¹Replace the placeholder URLs with the actual video links when preparing the final version of the report.

average, control inputs are reasonably smooth, and collisions are eliminated. RRT* and MPC offer advantages in trajectory smoothness and control effort, respectively, but at the cost of higher computation and more frequent safety interventions, while PID serves primarily as a simple reference design.

Future work should focus on extending this framework toward more realistic and challenging scenarios. On the planning side, incorporating dynamic replanning and prediction would enable safe navigation among moving obstacles without resorting solely to reactive CBF corrections. On the control and safety side, integrating estimator-aware tuning, adaptive or learning-based CBF parameters, and tighter coupling between MPC and safety constraints could reduce activation rates and further improve smoothness. Finally, validating the full stack on real hardware, under degraded sensing and in a variety of indoor and outdoor environments, will be essential to confirm that the observed performance—and especially the safety guarantees—carry over from simulation to practice.

REFERENCES

- [1] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge University Press, 2006.
- [2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [3] T. I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*. Chichester, U.K.: John Wiley & Sons, 2011.
- [4] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London, U.K.: Springer, 2010.
- [5] D. E. Kirk, *Optimal Control Theory: An Introduction*. Mineola, NY, USA: Dover Publications, 2004.
- [6] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Madison, WI, USA: Nob Hill Publishing, 2017.
- [7] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *Proc. 18th Eur. Control Conf. (ECC)*, Naples, Italy, 2019, pp. 3420–3431.
- [8] P. Ong, B. Capelli, L. Sabattini, and J. Cortés, “Nonsmooth control barrier function design of continuous constraints for network connectivity maintenance,” *Automatica*, vol. 156, art. 111209, Oct. 2023.
- [9] S. Heshmati-Alamdari, G. C. Karras, P. Marantos, and K. J. Kyriakopoulos, “A robust predictive control approach for underwater robotic vehicles,” *IEEE Trans. Control Syst. Technol.*, vol. 28, no. 6, pp. 2352–2363, 2020.